

A Project Report

on

Android Singleplayer Game

Submitted by

Mr. Vedant Ajay Mohite

in partial fulfillment for the award of the degree

of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

Under the guidance of

Mr. Sachin S. Bhostekar

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE



JANATA SHIKSHAN MANDAL's

Smt. Indirabai G. Kulkarni Arts, J. B. Sawant Science College &

Sau. Jankibai Dhondo Kunte Commerce College

(Affiliated to University of Mumbai)

ALIBAUG, 402 201

MAHARASHTRA

SEM VI

Academic Year: 2025-26



JANATA SHIKSHAN MANDAL's

Smt. Indirabai G. Kulkarni Arts, J. B. Sawant Science College & Sau. Jankibai

Dhondo Kunte Commerce College

(Affiliated to University of Mumbai)

ALIBAUG – MAHARASHTRA - 402 201

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that **Mr. Vedant Ajay Mohite** student of **T.Y.B.Sc. CS (Sem VI)** class has satisfactorily completed the project entitled, "**Android Single Player Game**", to be submitted in partial fulfillment for the award of degree of **BACHELOR OF SCIENCE in Computer Science** during the academic year **2025-26**.

Date of submission:

Project Guide

Head/ Incharge
Department of Computer Science

Signature of Examiner

DECLARATION

I, **Mr. Vedant Ajay Mohite**, hereby declare that the project entitled, “Food Donation & Surplus Re-distribution” to be submitted in partial fulfillment for the award of degree of **BACHELOR OF SCIENCE** in **Computer Science** during the academic year **2025-26** is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles

Signature of the Student:

Place:

Date:

ACKNOWLEDGEMENT

Every research endeavor, regardless of the field, requires the expert guidance and insights from knowledgeable individuals. I am immensely pleased to express my wholehearted thanks to my Project Coordinator, **Mrs. Shilpa N. Kawale**, for her exceptional advice, enthusiastic suggestions, highly valuable support and encouragement throughout this research of the project. I am also delighted to extend my heartfelt gratitude to my Project Guide, **Mr. Sachin S. Bhostekar**, for his significant contributions to the project. His dedication, expertise, and innovative approach were pivotal in driving the project to success.

I wish to convey my profound gratitude to Principal, **Dr. Mrs. Sonali Patil**, and our CS Department In-Charge, **Mrs. Shilpa N. Kawale**, for their sincere help and constant encouragement.

I am also thankful to our IT Department In-Charge, **Mr. Satyajit G. Tulpule**, and all the department staff for their years of experience, excellent support, and valuable suggestions.

Each of you brought unique skills and perspectives that enriched the development of my Food Donation & Surplus Re-distribution. Your contributions have ensured not only the effectiveness and reliability of the solution but have also set a high standard for future initiatives. I anticipate continuing our collaborative journey, building upon this success, and reaching even greater achievements.

Table of Contents

CHAPTER 1: PRELIMINARY INVESTIGATION	8
1.1 Organizational Overview	8
1.2 Description of Present System	8
1.3 Limitations of Present System	9
1.4 Proposed system and its advantages	9
1.5 Feasibility Study	10
1.6 Stakeholders	10
1.7 Technology used	10
1.8 Gantt Chart	11
CHAPTER 2: SYSTEM ANALYSIS	12
2.1 Fact Finding Techniques	12
2.2 Event Table	13
2.3 Use Case Diagram	14
2.4 Entity Diagram	14
2.6 Class Diagram	15
2.7 Activity Diagram	15
2.8 Sequence Diagram	16
2.9 State Chart Diagram	16
2.10 Component Diagram	17
2.11 Package Diagram	17
2.12 Deployment Diagram	18
CHAPTER 3: SYSTEM DESIGN	19
3.1 Converting ERD to Tables	19
CHAPTER 4: SYSTEM CODING	21
4.1 Menu Tree	21

4.2 List of tables with attributes and	22
constraints	22
4.3 Program Description with Naming	24
Conventions	24
4.4 Test Cases	26
4.5 Screen Layouts & Report Layouts	27
CHAPTER 5: IMPLEMENTATION AND TESTING	35
5.1 Implementation Approaches	35
5.2 Coding Details and Code Efficiency	36
References	39



Plagiarism Checker X – Report

Originality Assessment

20%



Overall Similarity

Date: 28/02/2026

Matches: 876/2595 words

Sources: 60
required.

Remarks: Low similarity
detected, check with your
supervisor if changes are
required.

Verify Report:

[View Certificate Online](#)

CHAPTER 1: PRELIMINARY INVESTIGATION

1.1 Organizational Overview

The gaming industry has experienced rapid growth due to the increasing use of smartphones and mobile applications. Mobile games have become one of the most popular forms of entertainment among users of all age groups. Android is one of the most widely used mobile operating systems, which provides developers with an open platform to create interactive and engaging applications.

This project focuses on developing an **Android Singleplayer Game** that combines music with gameplay mechanics. The game generates levels based on the rhythm of music tracks and challenges the player to navigate through obstacles while maintaining synchronization with the music beats.

The main goal of the project is to design and develop an engaging Android game that offers dynamic gameplay, replayability, and user-friendly controls.

1.2 Description of Present System

Currently, many mobile games follow a fixed level structure where the player experiences the same gameplay each time. These systems have the following characteristics:

- Predefined levels and obstacles
- Static gameplay environments
- Limited interaction with music or audio
- Less replay value

In many rhythm-based games, levels are manually designed by developers, which limits the number of levels available to players.

1.3 Limitations of Present System

The existing gaming systems have several limitations:

1. Static level generation.
2. Limited number of levels.
3. Lack of dynamic gameplay experience.
4. Low replayability.
5. Manual level design required by developers.
6. Less interaction between music and game environment.

These limitations reduce the long-term engagement of players.

1.4 Proposed system and its advantages

The proposed system is an **Android Singleplayer Game** where game levels are generated based on music beats.

Features

- Music-based level generation
- Dynamic obstacles
- Player jumping mechanics
- Score tracking
- Custom music support

In this system, the player controls a character that jumps across buildings or obstacles. The environment and obstacle patterns are generated based on the beat structure of the background music.

Advantages

1. Dynamic gameplay experience
2. Higher replayability
3. Unique level generation
4. Music synchronization with gameplay
5. Engaging user interface

Each time the player plays the game, the level appearance may vary even when the same music is used.

1.5 Feasibility Study

Technical Feasibility

The system is technically feasible because the technologies required for development such as **Android Studio, Kotlin, and libGDX** are widely available and supported.

Economic Feasibility

The development cost is minimal as open-source tools and libraries are used.

Operational Feasibility

The game is simple to use and requires minimal training for users.

1.6 Stakeholders

The stakeholders involved in this project include:

1. **Developer** – Responsible for designing and implementing the game.
2. **End Users** – Players who download and play the game.
3. **Testing Team** – Individuals responsible for testing functionality.
4. **Academic Institution** – Evaluates the project as part of academic requirements.
- 5.

1.7 Technology used

Technology	Description
Android Studio	IDE used for development
Kotlin	Programming language
libGDX	Game development framework
Gradle	Build automation tool

OpenGL	Rendering graphics
Git	Version control system

1.8 Gantt Chart

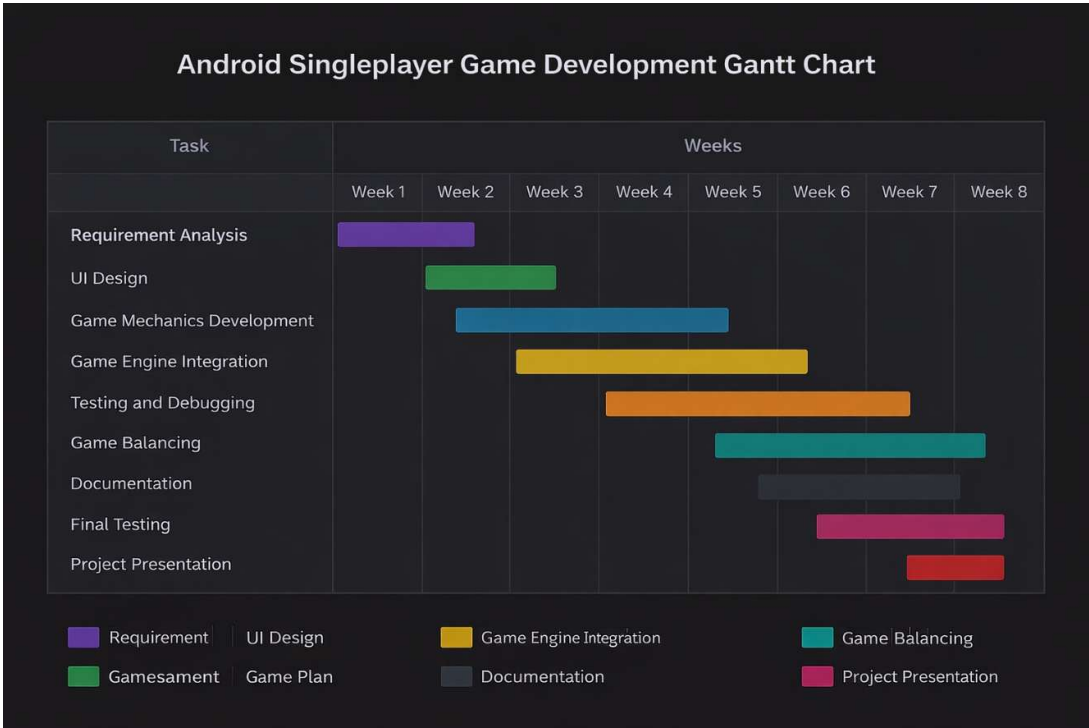


Figure 1 Gantt Chart

CHAPTER 2: SYSTEM ANALYSIS

2.1 Fact Finding Techniques

The following fact-finding techniques were used to gather requirements and understand system needs:

1. Literature Survey

- Studied multiplayer game architectures
- Reviewed Unity multiplayer documentation
- Analyzed Photon PUN networking model

2. Observation

- Observed popular Android multiplayer games
- Studied control schemes and UI patterns

3. Requirement Analysis

- Identified core multiplayer requirements:
 - Player synchronization
 - Matchmaking
 - Real-time actions
-
- Mobile performance constraints

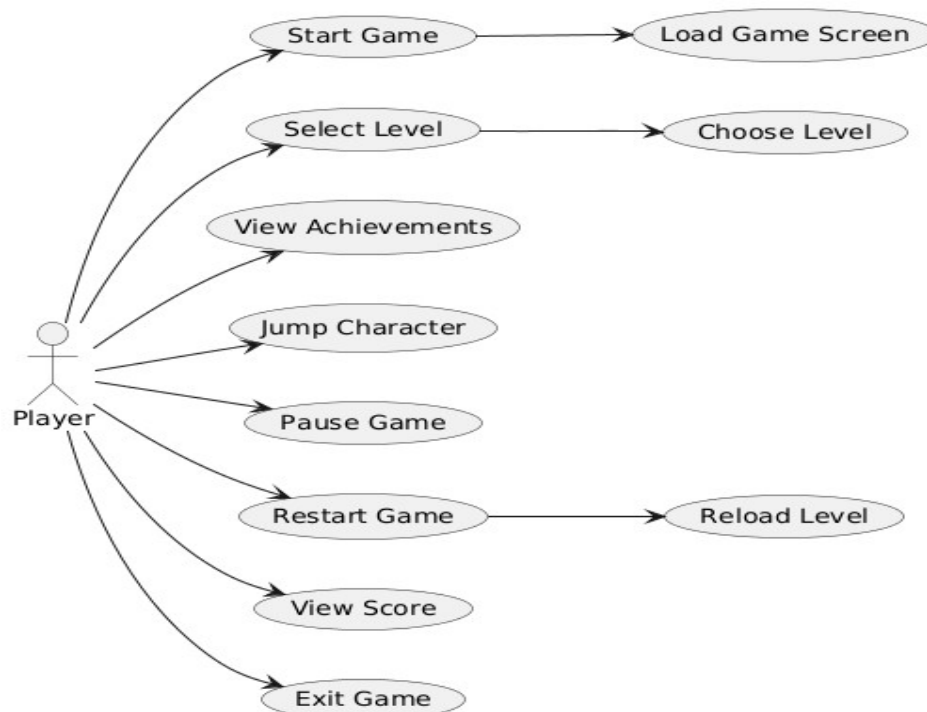
4. Technical Feasibility Study

- Evaluated Unity + Photon free-tier limits
- Verified Android device compatibility

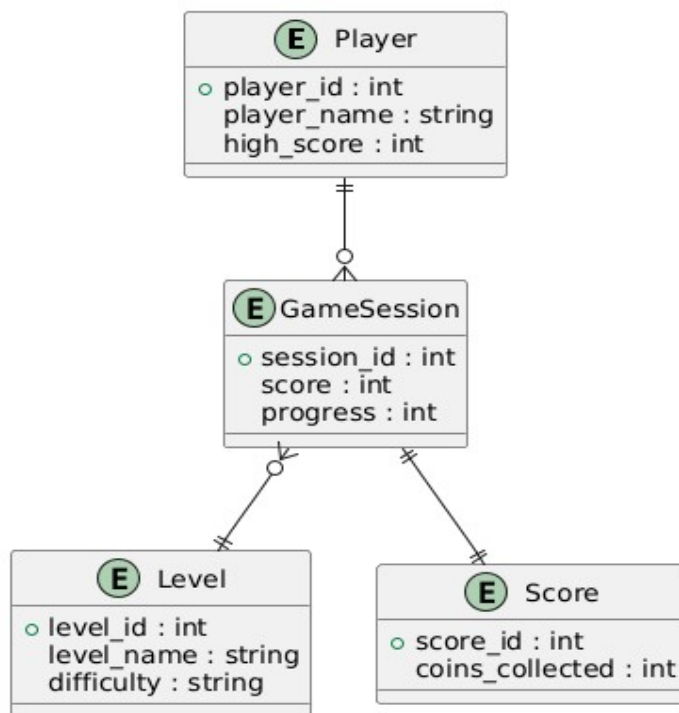
2.2 Event Table

Event No	Event Name	Trigger	Source	Response
E1	Player Login	App launch	Player	Connect to Photon server
E2	Create Room	Button click	Player	New multiplayer room created
E3	Join Room	Button click	Player	Player added to room
E4	Player Spawn	Room joined	System	Player prefab instantiated
E5	Player Move	Joystick input	Player	Position synced across network
E6	Player Shoot	Shoot button	Player	Damage applied to target
E7	Player Dash	Dash button	Player	Dash force applied
E8	Player Death	Health = 0	System	Player eliminated
E9	Match End	Last player alive	System	Winner declared

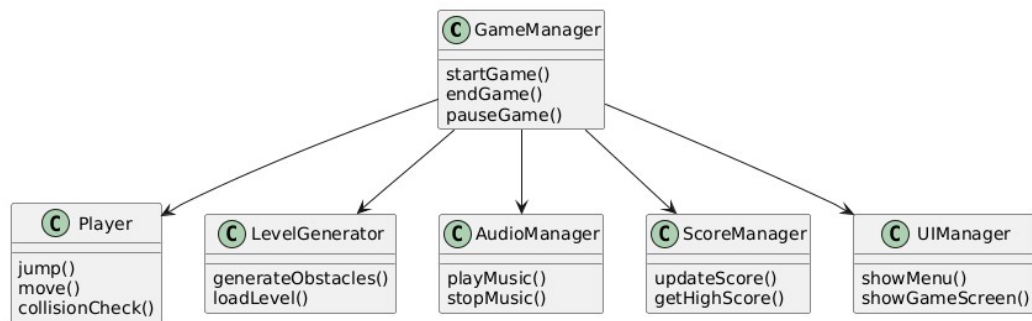
2.3 Use Case Diagram



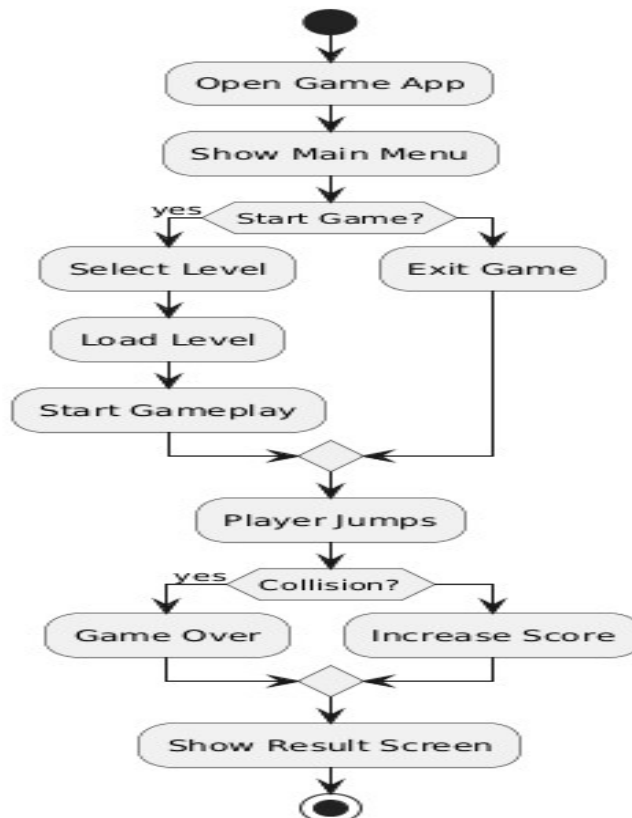
2.4 Entity Diagram



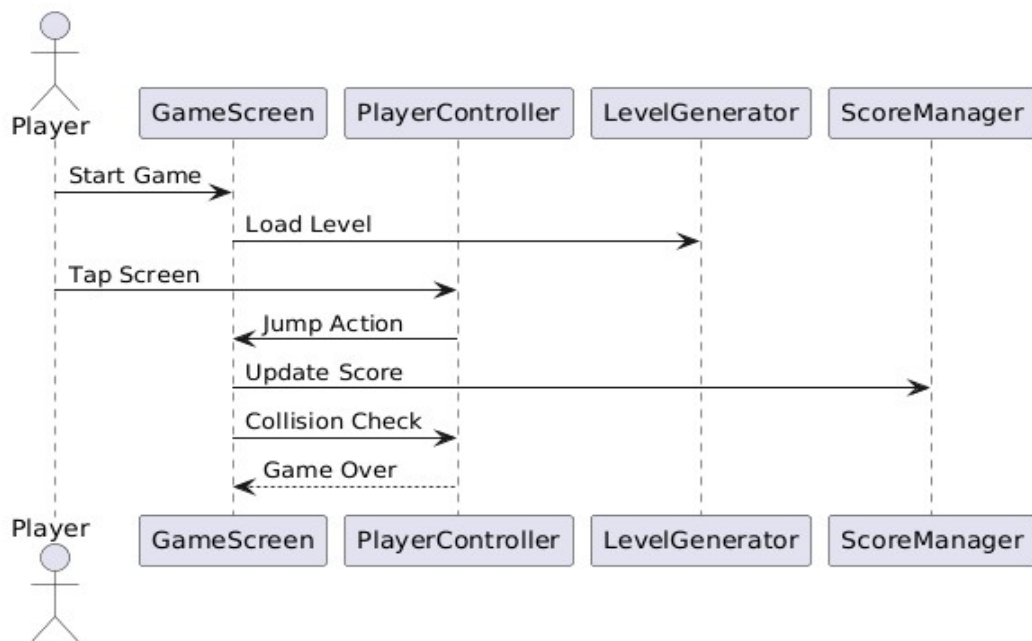
2.6 Class Diagram



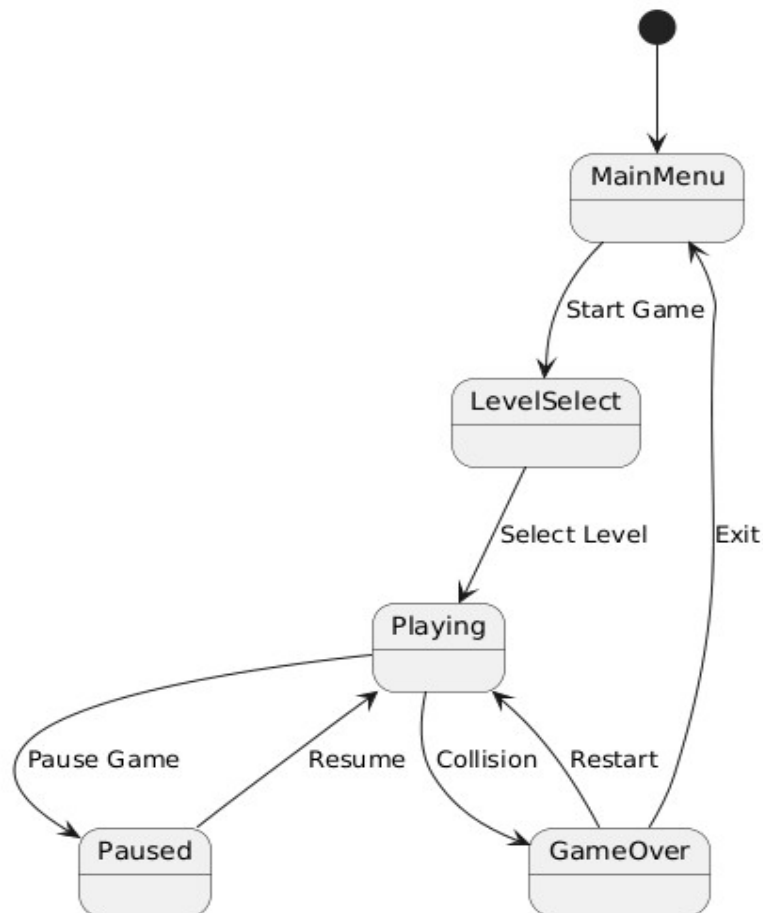
2.7 Activity Diagram



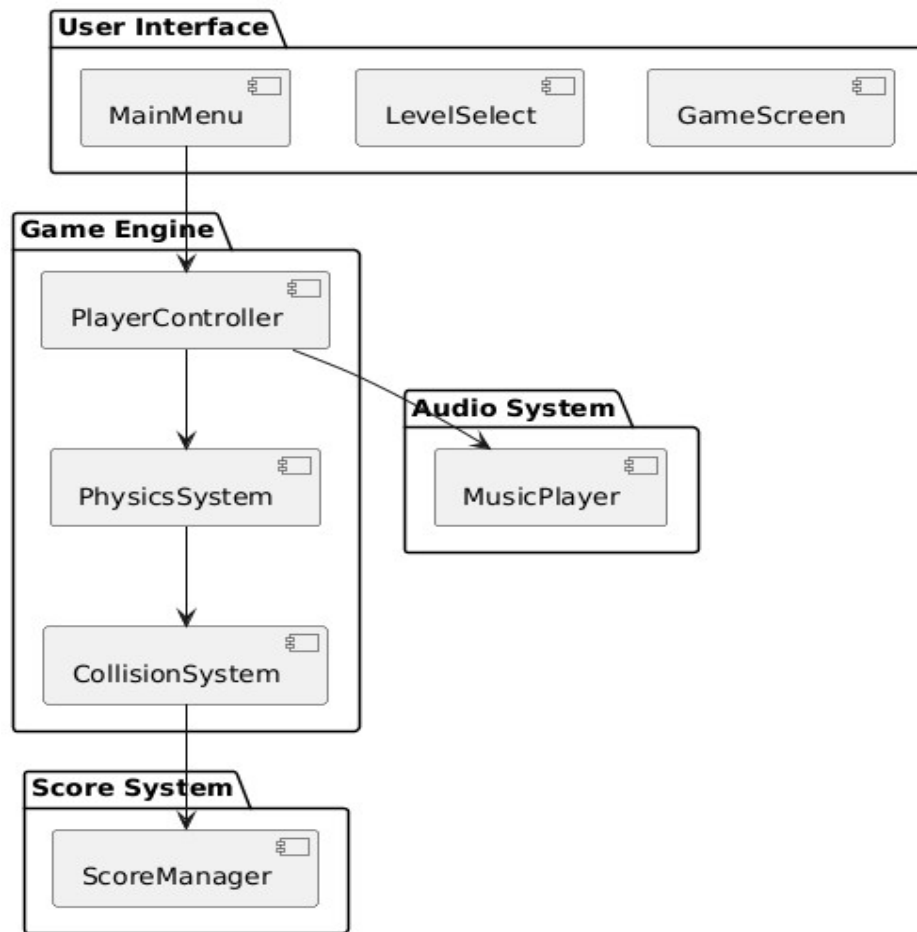
2.8 Sequence Diagram



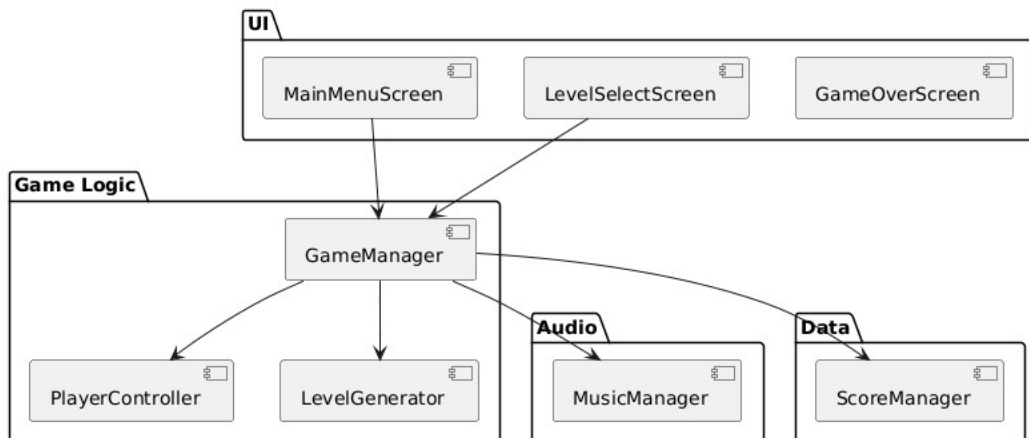
2.9 State Chart Diagram



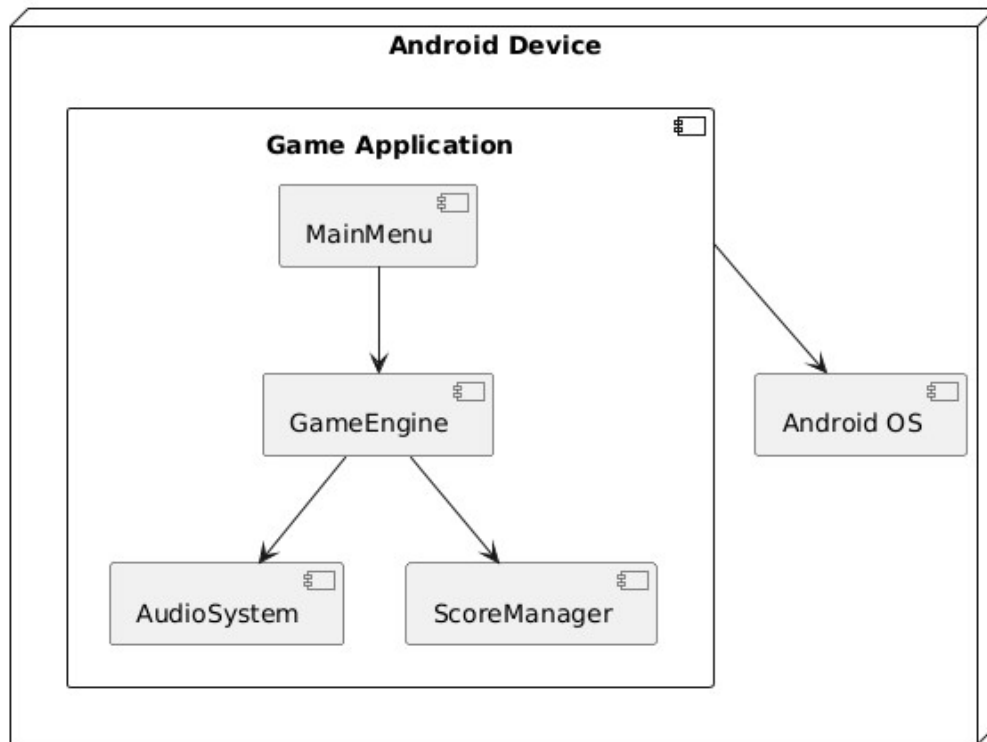
2.10 Component Diagram



2.11 Package Diagram



2.12 Deployment Diagram



CHAPTER 3: SYSTEM DESIGN

3.1 Converting ERD to Tables

1.Player Table

Field	Type
PlayerID	INT
PlayerName	VARCHAR
HighScore	INT

2.GameSession Table

Field	Type
SessionID	INT
PlayerID	INT
Score	INT
Level	INT

3.Music Table

Field	Type
MusicID	INT
FileName	VARCHAR
BeatData	TEXT

4. Scoreboard Table

Stores ranking of player scores.

Attribute	Data Type
score_id	INTEGER
player_id	INTEGER
score	INTEGER
rank	INTEGER
achieved_date	DATETIME

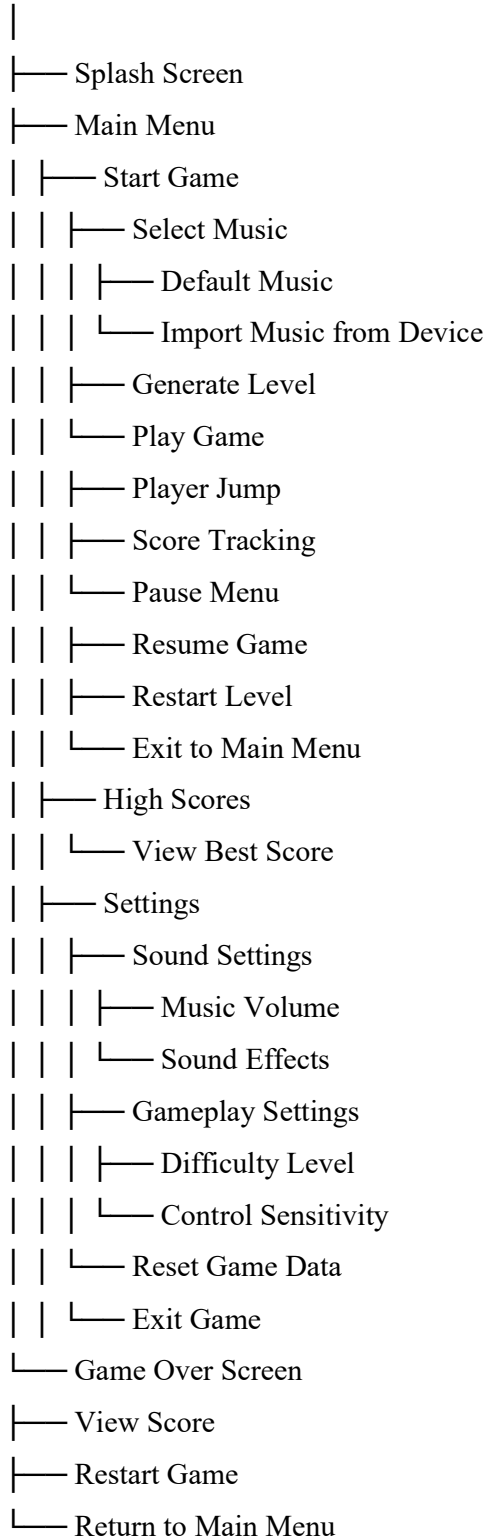
Foreign Key Constraint

player_id → references Player(player_id)

CHAPTER 4: SYSTEM CODING

4.1 Menu Tree

Application Start



4.2 List of tables with attributes and constraints

1. Player Table

Attribute	Data Type	Constraints	Description
player_id	INTEGER	Primary Key, Auto Increment	Unique ID of player
player_name	VARCHAR(50)	NOT NULL	Name of the player
high_score	INTEGER	DEFAULT 0	Highest score achieved
created_at	DATETIME	NOT NULL	Player profile creation date

2. Game_Session Table

Attribute	Data Type	Constraints	Description
session_id	INTEGER	Primary Key, Auto Increment	Unique session ID
player_id	INTEGER	Foreign Key	Reference to Player
score	INTEGER	NOT NULL	Score achieved in session
level_reached	INTEGER	DEFAULT 1	Level reached by player
duration	INTEGER	NOT NULL	Game duration in seconds
played_at	DATETIME	NOT NULL	Date and time of session

Foreign Key Constraint

player_id → references Player(player_id)

3. Music Table

Attribute	Data Type	Constraints	Description
music_id	INTEGER	Primary Key, Auto Increment	Unique music ID
music_name	VARCHAR(100)	NOT NULL	Name of music track
file_path	TEXT	NOT NULL	Storage path of music
bpm	INTEGER	NULL	Beats per minute
duration	INTEGER	NOT NULL	Length of music track

4. Game_Settings Table

Attribute	Data Type	Constraints	Description
setting_id	INTEGER	Primary Key, Auto Increment	Unique setting ID
music_volume	INTEGER	DEFAULT 50	Background music volume
sound_effect_volume	INTEGER	DEFAULT 50	Sound effect volume
difficulty_level	VARCHAR(20)	DEFAULT 'Normal'	Difficulty setting

control_sensitivity	INTEGER	DEFAULT 5	Player control sensitivity
----------------------------	---------	-----------	----------------------------

Foreign Key Constraint

player_id → references Player(player_id)

5. Scoreboard Table

Stores ranking of player scores.

Attribute	Data Type	Constraints	Description
score_id	INTEGER	Primary Key, Auto Increment	Unique score record
player_id	INTEGER	Foreign Key	Reference to player
score	INTEGER	NOT NULL	Score achieved
rank	INTEGER	NOT NULL	Ranking position
achieved_date	DATETIME	NOT NULL	Date of score achievement

Foreign Key Constraint

player_id → references Player(player_id)

4.3 Program Description with Name Conventions

Program Description

The **Android Singleplayer Game** is developed using an object-oriented and modular programming approach. The program is divided into different modules such as game control, player actions, level generation, audio management, and score tracking.

When the application starts, the main menu screen is displayed. After starting the game, the system loads the selected music and generates obstacles dynamically. The player controls the character by performing jumps to avoid obstacles. During gameplay, the system continuously checks for player input, collision detection, and score updates. When a collision occurs, the game ends and the final score is displayed.

Main modules of the program include:

- **Game Manager** – Controls game flow such as start, pause, and end.
- **Player Controller** – Handles player movement and jumping.
- **Level Generator** – Generates obstacles based on music beats.
- **Audio Manager** – Handles music playback and beat synchronization.
- **Score Manager** – Tracks and updates player score.
- **User Interface** – Manages game screens and menus.

Name Convention

To maintain consistency and readability in the code, standard naming conventions are used.

Class Names (PascalCase)

Example:

`GameManager, PlayerController, LevelGenerator`

Variables (camelCase)

Example:

`playerScore, currentLevel, musicVolume`

Methods (camelCase)

Example:

`startGame(), pauseGame(), updateScore()`

Constants (UPPER_CASE)

Example:

MAX_SCORE, DEFAULT_VOLUME, GRAVITY_FORCE

Package Names (lowercase)

Example:

game.ui, game.audio, game.player

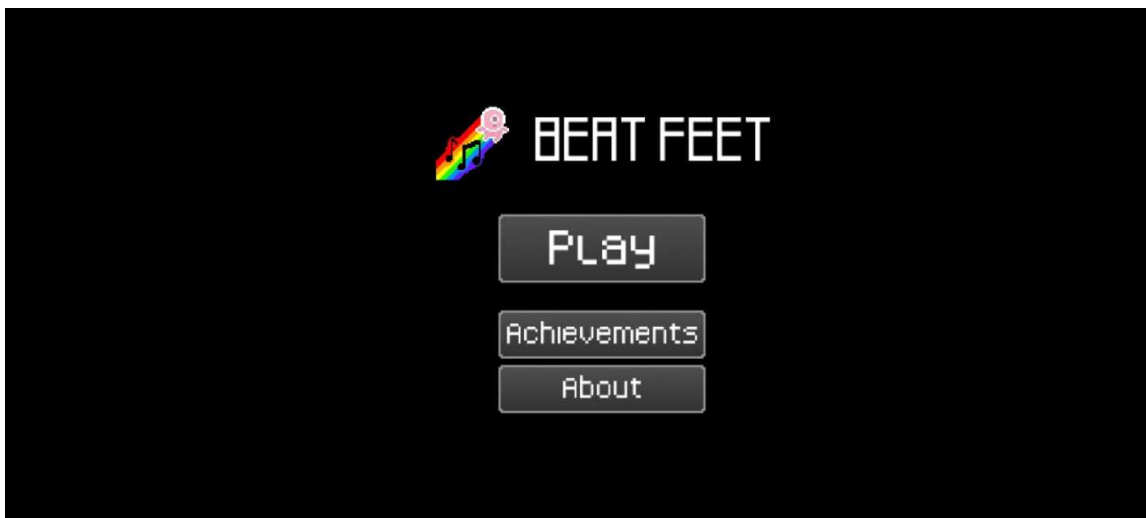
4.4 Test Cases

Test Case ID	Test Case Description	Input / Action	Expected Result	Status
TC01	Launch Application	Open the game application	Splash screen and main menu should appear successfully	Pass
TC02	Start New Game	Click Start Game button	Game screen loads and gameplay begins	Pass
TC03	Player Jump Action	Tap screen during gameplay	Player character jumps to avoid obstacle	Pass
TC04	Collision Detection	Player hits an obstacle	Game ends and Game Over screen appears	Pass
TC05	Score Update	Player successfully avoids obstacles	Player score increases during gameplay	Pass
TC06	Pause Game	Press Pause button during gameplay	Game pauses and pause menu appears	Pass
TC07	Resume Game	Click Resume from pause menu	Gameplay continues from paused state	Pass
TC08	Restart Game	Click Restart after game over	Game restarts from beginning	Pass

TC09	View High Score	Open High Score menu	Highest score is displayed correctly	Pass
TC10	Exit Game	Select Exit from main menu	Game application closes successfully	Pass

4.5 Screen Layouts & Report Layouts

Main Screen



```
class MainMenuScreen(val game: MyGame) : ScreenAdapter() {
    private lateinit var stage: Stage
    private lateinit var skin: Skin
    override fun show() {
        stage = Stage(ScreenViewport())
        Gdx.input.inputProcessor = stage
        skin = Skin(Gdx.files.internal("uiskin.json"))
        val table = Table()
        table.setFillParent(true)
        table.center()
        // Title
        val title = Label("ANDROID SINGLEPLAYER GAME", skin)
```

```

title.setFontScale(2f)

// Buttons
val playButton = TextButton("Play", skin)
val achievementButton = TextButton("Achievements", skin)
val aboutButton = TextButton("About", skin)
table.add(title).padBottom(40f)
table.row()
table.add(playButton).width(250f).height(60f).pad(10f)
table.row()
table.add(achievementButton).width(250f).height(60f).pad(10f)
table.row()
table.add(aboutButton).width(250f).height(60f).pad(10f)
stage.addActor(table)

// Button Listeners
playButton.addListener(object : ClickListener() {
    override fun clicked(event: InputEvent?, x: Float, y: Float) {
        game.screen = GameScreen(game)
    }
})

achievementButton.addListener(object : ClickListener() {
    override fun clicked(event: InputEvent?, x: Float, y: Float) {
        println("Achievements Clicked")
    }
})

aboutButton.addListener(object : ClickListener() {
    override fun clicked(event: InputEvent?, x: Float, y: Float) {
        println("About Clicked")
    }
})
}

override fun render(delta: Float) {
    Gdx.gl.glClearColor(0f, 0f, 0f, 1f)
    Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT)
}

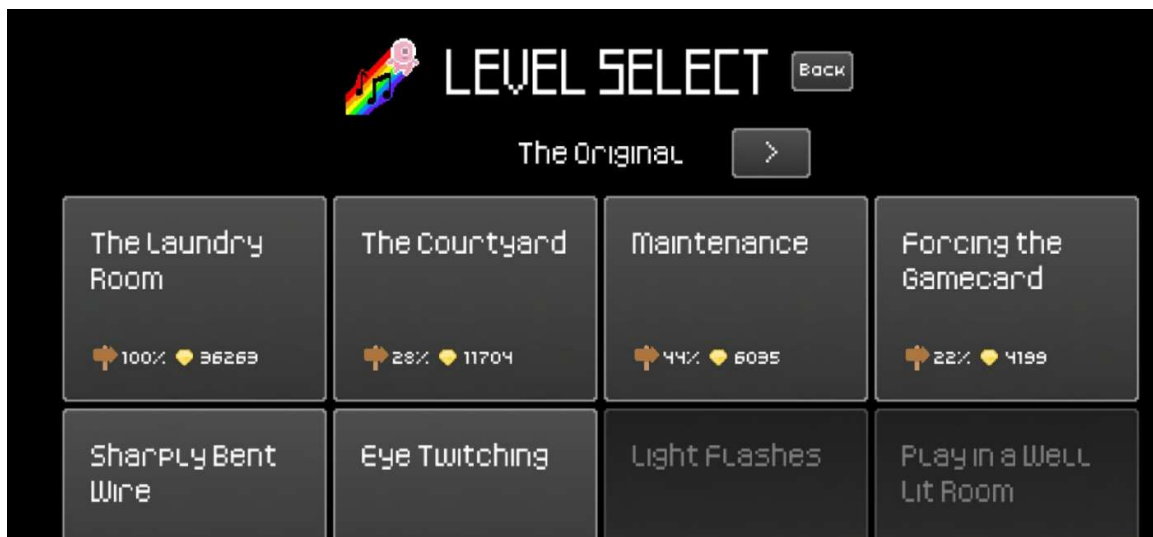
```

```

        stage.act(delta)
        stage.draw()
    }
    override fun dispose() {
        stage.dispose()
    }
}

```

Level Select Screen : -



```

class LevelSelectScreen(val game: MyGame) : ScreenAdapter() {
    private lateinit var stage: Stage
    private lateinit var skin: Skin
    override fun show() {
        stage = Stage(ScreenViewport())
        Gdx.input.inputProcessor = stage
        skin = Skin(Gdx.files.internal("uiskin.json"))
        val mainTable = Table()
        mainTable.setFillParent(true)
        mainTable.top()
        // Header
        val headerTable = Table()
        val title = Label("LEVEL SELECT", skin)
        title.setFontScale(2f)
    }
}

```

```

val backButton = TextButton("Back", skin)
headerTable.add(title).expandX().left().pad(20f)
headerTable.add(backButton).right().pad(20f)
mainTable.add(headerTable).expandX().fillX()
mainTable.row()

// Category selector
val categoryTable = Table()
val categoryLabel = Label("The Original", skin)
val nextButton = TextButton(">", skin)
categoryTable.add(categoryLabel).pad(20f)
categoryTable.add(nextButton).pad(20f)
mainTable.add(categoryTable)
mainTable.row()

// Level Grid
val gridTable = Table()
val levels = arrayOf(
    "The Laundry Room",
    "The Courtyard",
    "Maintenance",
    "Forcing the Gamecard",
    "Sharply Bent Wire",
    "Eye Twitching",
    "Light Flashes",
    "Play in a Well Lit Room"
)

var count = 0
for (level in levels) {
    val levelButton = TextButton(level, skin)
    gridTable.add(levelButton)
        .width(300f)
        .height(150f)
        .pad(10f)
    count++
    if (count % 4 == 0) {

```

```

        gridTable.row()
    }
}
mainTable.add(gridTable).padTop(30f)

stage.addActor(mainTable)

// Back Button
backButton.addListener(object : ClickListener() {
    override fun clicked(event: InputEvent?, x: Float, y: Float) {
        game.screen = MainMenuScreen(game)
    }
}))
}

override fun render(delta: Float) {

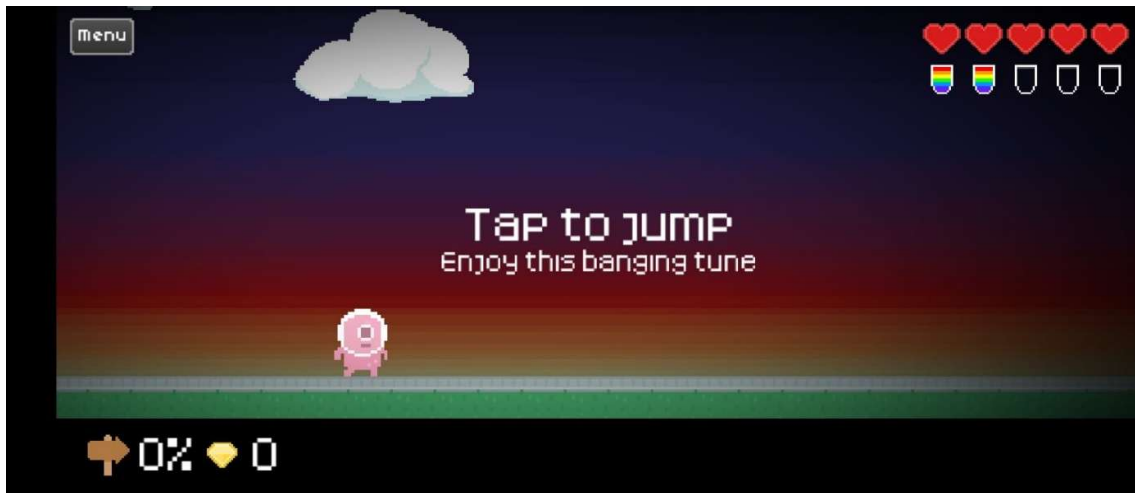
    Gdx.gl.glClearColor(0f, 0f, 0f, 1f)
    Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT)

    stage.act(delta)
    stage.draw()
}

override fun dispose() {
    stage.dispose()
}
}

```

Game Start Screen:-



```
class GameScreen(val game: MyGame) : ScreenAdapter() {  
    private lateinit var batch: SpriteBatch  
    private lateinit var stage: Stage  
    private lateinit var font: BitmapFont  
    private lateinit var player: Texture  
    private lateinit var cloud: Texture  
    private lateinit var ground: Texture  
    private lateinit var heart: Texture  
  
    private var playerX = 200f  
    private var playerY = 150f  
    override fun show() {  
        batch = SpriteBatch()  
        stage = Stage(ScreenViewport())  
        font = BitmapFont()  
        player = Texture("player.png")  
        cloud = Texture("cloud.png")  
        ground = Texture("ground.png")  
        heart = Texture("heart.png")  
        // Menu Button  
        val skin = Skin(Gdx.files.internal("uiskin.json"))  
        val menuButton = TextButton("Menu", skin)
```

```

        menuButton.setPosition(20f, Gdx.graphics.height - 70f)
        menuButton.addListener(object : ClickListener() {
            override fun clicked(event: InputEvent?, x: Float, y: Float) {
                game.screen = MainMenuScreen(game)
            }
        })
        stage.addActor(menuButton)
        Gdx.input.inputProcessor = InputMultiplexer(stage, object : InputAdapter() {
            override fun touchDown(screenX: Int, screenY: Int, pointer: Int, button: Int):
Boolean {
                jump()
                return true
            }
        })
    }
    private fun jump() {
        playerY += 120
    }
    override fun render(delta: Float) {
        Gdx.gl.glClearColor(0.1f, 0.1f, 0.2f, 1f)
        Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT)
        batch.begin()
        // Cloud
        batch.draw(cloud, 400f, 450f)
        // Ground
        batch.draw(ground, 0f, 120f)
        // Player
        batch.draw(player, playerX, playerY)
        // Instruction Text
        font.data.setScale(3f)
        font.draw(batch, "Tap to Jump", 420f, 350f)
        font.data.setScale(1.5f)
        font.draw(batch, "Enjoy this banging tune", 420f, 300f)
        // Hearts (lives)

```

```
        for (i in 0 until 5) {
            batch.draw(heart, Gdx.graphics.width - (i * 60f) - 60f, Gdx.graphics.height -
60f)
        }
        // Progress
        font.draw(batch, "0% 0", 50f, 80f)
        batch.end()
        stage.act()
        stage.draw()
    }
    override fun dispose() {
        batch.dispose()
        player.dispose()
        cloud.dispose()
        ground.dispose()
        heart.dispose()
    }
}
```

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The Android Singleplayer Game was implemented using a modular and structured development approach. The system was developed in stages to ensure proper functionality, testing, and performance.

1. Modular Development

The application was divided into different modules such as the main menu, level selection, gameplay screen, score system, and game over screen. Each module was developed independently and later integrated into the complete system.

2. Object-Oriented Programming

The system was implemented using object-oriented programming principles. Different classes were created for game components such as the player, obstacles, level generator, and audio manager to improve code reusability and maintainability.

3. Incremental Development

The game was developed incrementally by implementing core features first, such as player movement and jumping mechanics. Additional features like obstacle generation, scoring system, and user interface elements were added gradually.

4. Testing and Debugging

Each module was tested individually to ensure correct functionality. Bugs and performance issues were identified and fixed during the development process to improve stability and gameplay experience.

5. User Interface Implementation

The user interface was implemented using game development frameworks and libraries. Screens such as the main menu, level selection, gameplay screen, and game over screen were designed to provide smooth navigation and interaction.

5.2 Coding Details and Code Efficiency

The Android Singleplayer Game was developed using efficient coding practices to ensure smooth performance and maintainability of the application. The system was implemented using structured programming and object-oriented design principles.

Coding Details

The game was developed using Kotlin programming language and a game development framework to handle graphics rendering, user input, and game logic. Different classes were created to manage various components of the game such as player movement, obstacle generation, level handling, and score tracking.

The code is organized into separate modules including:

- **Game Manager** – Controls the overall game flow.
- **Player Controller** – Handles player movement and jumping mechanics.
- **Level Generator** – Creates obstacles and level patterns.
- **Audio Manager** – Manages background music and sound effects.
- **User Interface Module** – Handles menus, buttons, and screen navigation.

This modular structure improves code readability and simplifies future updates or modifications.

Code Efficiency

Several techniques were used to improve the efficiency of the code:

- **Object-Oriented Design** was used to reuse classes and reduce code duplication.
- **Efficient Rendering Techniques** were implemented to ensure smooth graphics performance during gameplay.
- **Optimized Collision Detection** was used to accurately detect interactions between the player and obstacles.
- **Memory Management** techniques were applied to release unused resources and avoid memory leaks.
- **Event-based Input Handling** was used to respond quickly to player actions such as tapping to jump.

References

Android Developers. Android Developer Documentation. Available at:

<https://developer.android.com>

libGDX Developers. libGDX Game Development Framework Documentation.

Available at:

<https://libgdx.com>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).

Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Pressman, R. S. (2014).

Software Engineering: A Practitioner's Approach. McGraw-Hill Education.

Sommerville, I. (2016).

Software Engineering (10th Edition). Pearson Education.

Google. Kotlin Programming Language Documentation. Available at:

<https://kotlinlang.org/docs>

W3Schools. SQL and Database Design Tutorials. Available at:

<https://www.w3schools.com>

PlantUML. PlantUML Documentation for UML Diagrams. Available at:

<https://plantuml.com>

Android Open Source Project. Android Architecture and Development Guide.

Available at:

<https://source.android.com>

Game Development Resources. Mobile Game Design and Development Concepts.

Available at:

<https://gamedevelopment.tutsplus.com>

